

AI-Powered Sign Language Recognition Presentation Script

Prepared for the 20-slide ASL presentation deck

Speakers: Abrar Mustafa and Mohammad Ashad Khan

Target total time: about 22 minutes. Abrar: about 11 minutes. Mohammad: about 11 minutes.

Speaker	Main responsibility	Approx. time
Abrar	Opening, problem, dataset, preprocessing, Basic CNN, training setup, quantitative results, qualitative summary.	~11:00
Mohammad	System pipeline, transfer-learning models, Streamlit deployment, gesture actions, assumptions, runtime efficiency, operating conditions.	~11:00

Timing note

The script is intentionally a little over 20 minutes so you have room to speak naturally. If your instructor asks for exactly 20 minutes, shorten the opening and conclusion by about one minute total.

Fairness note

The three-model section is split by responsibility: Abrar explains the custom Basic CNN and the quantitative comparison, while Mohammad explains MobileNetV2, EfficientNetB0, and why MobileNetV2 is the default deployment choice. This keeps both workload and speaking time balanced.

Run of Show

Slide	Topic	Speaker	Time
1	Title	Abrar	0:45
2	Presentation Outline	Abrar	0:35
3	The Problem We Are Solving	Abrar	1:05
4	Why This Matters	Abrar	0:55
5	Project Objectives	Abrar	0:55
6	Dataset Overview	Abrar	1:05
7	Data Characteristics	Abrar	1:05
8	Preprocessing and Augmentation	Abrar	1:10
9	Solution Overview	Mohammad	1:10
10	Hand Detection and Representation	Mohammad	1:10
11	Model Architectures	Abrar	1:35
11b	Model Architectures - Transfer Learning	Mohammad	1:35
12	Training and Evaluation Setup	Abrar	1:05
13	Streamlit Application	Mohammad	1:10
14	Gesture-Based Action Mapping	Mohammad	1:05
15	Key Assumptions	Mohammad	1:05
16	Effectiveness Results	Abrar	1:20
17	Runtime Performance	Mohammad	1:10
18	Operating Conditions	Mohammad	1:05
19	Qualitative Observations	Abrar	1:00
20	Conclusion and Future Work	Both	1:15

Slide-by-Slide Script

Slide 1: Title

Abrar - 0:45

Good morning everyone. We are Abrar Mustafa and Mohammad Ashad Khan, and our project is called AI-Powered Sign Language Recognition and Gesture-Based Action Control. The main idea is simple: we wanted to see whether a normal webcam, a deep learning model, and a web application could recognize ASL alphabet signs in real time and then use those recognized gestures to perform useful actions. So this is not only a classification project. It is also a small intelligent consumer technology system: it senses a human gesture, learns to classify it, and then acts by generating text or opening a web service.

Slide 2: Presentation Outline

Abrar - 0:35

Here is the structure of our presentation. First, I will introduce the problem and why it matters. Then I will explain the dataset and preprocessing pipeline, including why we used 24 classes instead of all 26 letters. After that, we will move into the solution design, the three models, deployment, action control, assumptions, results, operating conditions, and finally our conclusion and future work. We divided the explanation so both of us cover a balanced part of the system: I will focus more on the problem, data, preprocessing, and the custom CNN baseline. Mohammad will focus more on the transfer-learning models, deployment, runtime behavior, and practical observations.

Slide 3: The Problem We Are Solving

Abrar - 1:05

The problem we are trying to solve is accessibility in everyday technology. Sign language is essential for Deaf and hard-of-hearing communities, but most consumer technology still expects users to type, click, or touch a screen. That creates a gap between natural gesture communication and the way digital systems normally work. Existing sign recognition systems can also be limited because they may require special sensors, controlled lab conditions, or offline processing. Our challenge was to ask: can we use only a standard webcam and deep learning to recognize ASL finger-spelling in real time, and then connect that recognition to real digital actions such as writing text or opening web services?

Slide 4: Why This Matters

Abrar - 0:55

This matters because ASL finger-spelling is used for names, places, technical terms, and words that may not have a common sign. If technology can recognize those signs more naturally, it can become more inclusive. The consumer technology part is also important. We did not want a system that only predicts labels inside a notebook. We wanted to connect sensing, learning, and acting. The webcam senses the hand, the neural network predicts the letter, and the application uses that prediction to support text generation or action control. That is why the project fits the ICT theme.

Slide 5: Project Objectives

Abrar - 0:55

Our objectives were fivefold. First, recognize static ASL alphabet signs from a live webcam feed. Second, preprocess the data and compare three models: a custom Basic CNN, MobileNetV2, and EfficientNetB0. Third, deploy everything in a Streamlit application so the system is interactive rather than only experimental. Fourth, map recognized gestures to text and web actions. Fifth, evaluate both effectiveness and efficiency. Effectiveness means validation accuracy and prediction quality, while efficiency means live FPS and responsiveness during real use.

Slide 6: Dataset Overview

Abrar - 1:05

For the dataset, we had 4,681 total images organized into 26 letter folders. However, the final training setup uses 24 classes: A through Y, excluding J and Z. We excluded J and Z because they are motion-based ASL signs. Our models classify static images, so including motion signs would be inconsistent with the input representation. The images are resized to 224 by 224 pixels, and the data split is 80 percent training and 20 percent validation. That gives us 3,458 training images and 863 validation images. This dataset structure made it suitable for a controlled multi-class classification task.

The dataset is also quite balanced. Most letters have around 180 images, and only a few classes such as Q and R have slightly fewer. This is useful because the model is not strongly biased toward one class just because it has more examples. Visually, the dataset contains static hand poses in RGB images. But an important limitation appears when we move to the live application. At runtime, the hand is converted into a landmark skeleton drawn on a white canvas. So there is a domain gap: the training images are real hand photos, while live predictions use landmark-style drawings. This gap helps explain why very high validation accuracy does not always mean perfect live performance.

The preprocessing pipeline is consistent across the notebooks, but with one model-specific difference. The Basic CNN and MobileNetV2 use pixel rescaling by dividing values by 255. EfficientNetB0 uses its dedicated `preprocess_input` function because EfficientNet expects a specific normalization format. For training, we use an 80/20 split with seed 42, batch size 32, Adam optimizer, and categorical cross-entropy loss. For the transfer-learning models we also use augmentation: small rotations, width and height shifts, shear, zoom, and horizontal flipping. The purpose is to make the models more robust to natural variation in hand position and camera framing.

Handoff cue: pause briefly, then let the next speaker connect this point to the next system layer.

Now I will continue with the system design. The full pipeline is: webcam input, hand detection, landmark canvas generation, CNN classification, stability filtering, and then either action control or text output. The webcam frame is captured with OpenCV. The hand detector finds the hand and the 21 landmarks. Those landmarks are drawn into a clean canvas, resized, and sent into the selected Keras model. After the model predicts a letter, the application does not immediately trigger an action. It first checks stability, meaning the same prediction must appear across several consecutive frames. This reduces accidental triggers and makes the system more usable.

In the recognition pipeline, we crop around the detected hand using padding so the hand is not cut off. Then we draw the 21 landmark points and their connections on a 400 by 400 white canvas. This canvas is resized to 224 by 224 to match the model input shape. The system processes one hand per frame, so multiple-hand situations are outside the project scope. We also use a confidence threshold of 80 percent and an action cooldown of 2.5 seconds. The stability filter is especially important because a single wrong frame should not open a browser tab or add a wrong character to the text buffer.

This slide shows the three models. I will cover the Basic CNN, and Mohammad will continue with the transfer-learning models. The Basic CNN is our custom baseline. It has three Conv2D blocks with increasing filters: 32, 64, and 128. After feature extraction, it uses a dense layer with 256 units, dropout of 0.5, and a final softmax layer with 24 outputs. It trained for 10 epochs. The reason we included this model was to have a from-scratch comparison point. It tells us how well a relatively simple architecture can learn the dataset without ImageNet pretraining. Surprisingly, on the validation set it performed extremely well, reaching about 99.30 percent validation accuracy. However, the saved model is much larger than the transfer-learning models, so accuracy is not the only factor when deciding what to deploy.

The other two models are MobileNetV2 and EfficientNetB0. Both use transfer learning with ImageNet weights and a frozen base network, then a custom classification head for our 24 ASL classes. MobileNetV2 is designed to be lightweight, using depthwise separable convolutions, so it is a good option for consumer applications. EfficientNetB0 is also efficient, but it uses compound scaling and its own preprocessing method. Both transfer-learning models trained for 15 epochs. MobileNetV2 became the default deployed model because it gives a strong balance between validation accuracy, model size, and practical speed. EfficientNetB0 also performed very well, reaching a peak validation accuracy of about 98.84 percent, but MobileNetV2 felt like the better default for interactive use.

Slide 12: Training and Evaluation Setup

Abrar - 1:05

The training was implemented in three Jupyter notebooks under the training folder. Each notebook saves its result as a Keras model file in the models folder. Since the batch size is 32 and the training set has 3,458 images, each epoch has about 109 batches. We evaluated effectiveness using validation accuracy and validation loss. We evaluated efficiency using FPS during live Streamlit inference. All benchmarks were done on CPU because native Windows TensorFlow from version 2.11 onward does not provide the same GPU support. This matters because our runtime FPS is strongly affected by CPU inference and Streamlit rerun overhead.

Handoff cue: pause briefly, then let the next speaker connect this point to the next system layer.

Slide 13: Streamlit Application

Mohammad - 1:10

For deployment, we built a single-page Streamlit app in app.py. The sidebar lets the user choose among the three models without restarting the application. The app shows the webcam preview, the detected landmark overlay, the current prediction, confidence, FPS, and the stable-frame counter. It also keeps an event history of the last triggered actions, which helps us see what the system actually did during testing. We included an ASL reference chart beside the camera feed, and we also built a prototype desktop version using Tkinter in run_mediapipe.py. Streamlit became the main deployment because it was easier to demonstrate and adjust interactively.

Slide 14: Gesture-Based Action Mapping

Mohammad - 1:05

The application supports two main modes. In Text Generation mode, every confirmed letter is appended to a running text buffer, so users can spell words letter by letter. In Web Actions mode, specific letters map to web services. For example, G can open Google, Y can open YouTube, and E can open Gmail. The letter S can search the generated text on Google. We added an action cooldown so the same gesture does not repeatedly trigger the same action. Text-to-speech is also listed as a dependency, so speech output is a natural future extension even though the current demonstration focuses on recognition and action control.

Slide 15: Key Assumptions

Mohammad - 1:05

The system depends on several assumptions. It recognizes static ASL alphabet signs, not full words, phrases, or motion signs like J and Z. It assumes one visible hand, reasonable centering, and enough light for the detector to work. It also assumes the user holds each sign steady long enough for the stability filter. Technically, we assumed that landmark skeletons generated from live video would be close enough to the training distribution. In practice, that assumption is only partly true. The system works, but the difference between training photos and live landmark drawings is one of the main causes of live confusion.

Slide 16: Effectiveness Results

Abrar - 1:20

The quantitative results are strong. The Basic CNN reached about 98.70 percent training accuracy and 99.30 percent validation accuracy with a validation loss of 0.030. MobileNetV2 reached about 95.46 percent training accuracy and 98.49 percent validation accuracy with a validation loss of 0.055. EfficientNetB0 reached a peak validation accuracy of about 98.84 percent and a validation loss around 0.065. So all three models exceed 98 percent validation accuracy. The important interpretation is that the dataset classification task was learned very successfully. But for live use, the stability filter and confidence threshold are still needed because live input is noisier than validation data.

Handoff cue: pause briefly, then let the next speaker connect this point to the next system layer.

Slide 17: Runtime Performance

Mohammad - 1:10

For efficiency, we measured FPS during Streamlit inference. The logged runs show average FPS values around 5 to 10 frames per second, with examples such as 4.96 FPS for a 30-second run and 10.19 FPS for a 60-second run. This is not true 30 FPS real-time video. But for this project, the user intentionally holds a sign for confirmation, so the system can still be usable even at lower FPS. The FPS varies because each frame involves webcam capture, hand detection, landmark drawing, preprocessing, model inference, and Streamlit interface updates. GPU acceleration, TensorFlow Lite conversion, or model optimization would likely improve responsiveness.

The system works best when lighting is even, the background is plain, one hand is fully visible, and the user holds the sign steady. It also works better when signs are close to the training poses and confidence stays above the threshold. It degrades when there are multiple hands, partial occlusions, low light, fast transitions, or heavy CPU load. Similar-looking signs such as M, N, and T can also be confused. And if the user attempts J or Z, the system is not designed for that because those require motion recognition rather than static classification.

From our team testing, the positive observations were that Web Actions mode feels intuitive, the model switcher works smoothly, and the live landmark overlay gives helpful feedback. Text Generation can spell short words when signs are held carefully. The confidence slider is also useful because it lets the user choose between precision and responsiveness. On the negative side, the stability requirement can make spelling feel slow, similar signs can still be confused, and CPU FPS makes the system feel less real-time than expected. We also did not conduct a formal user study, so our feedback is based on internal testing sessions.

Abrar: To conclude, we built a complete pipeline from dataset preparation to model training, real-time recognition, and gesture-driven actions. We compared three models and all achieved more than 98 percent validation accuracy on 24 ASL classes. The project shows how sign recognition can move beyond a notebook and become part of an interactive consumer technology system. Mohammad: For future work, the most important improvements are to add motion detection for J and Z, train directly on landmark-style images to reduce the domain gap, optimize the models for faster inference, and integrate text-to-speech. We would also like to expand from letters to words and phrases and conduct a formal usability study with real users. Thank you. We are happy to take questions.

Final Speaker Checklist

- Practice the handoffs after slides 8, 12, and 16 so the presentation feels shared rather than split in half.
- Keep the three models clear: Basic CNN is the custom baseline, MobileNetV2 is the default deployed model, and EfficientNetB0 is the second transfer-learning comparison.
- Mention the key limitation whenever results come up: validation accuracy is high, but live input has a domain gap because the app uses landmark skeletons.
- For a 20-minute target, speak at a calm pace and trim small examples if needed. For a 22-minute slot, use the full script.
- During questions, emphasize that the project is a complete ICT pipeline: sensing, learning, and acting.